

3

진도 (공식 교재 기준)

- 2-1_자연어 처리 기본
 - 워드 임베딩과 순환신경망 기반 모델 (RNN & LSTM)
 - 자연어 생성 모델 (Seq2Seq, Attention)
 - Transformer
- 3-1_딥러닝과 영상 모델-2
 - 2차시: 다양한 신경망 모델

강의자료

- 머신러닝 Lv.2
 - 1. 순환신경망
 - 2. 트랜스포머
- 머신러닝 Lv.2 > 레포트
 - 1. RNN 보고서
 - 2. GAN + Transformer 보고서
- 자연어처리
 - 1. Recurrent Neural Networks
 - 2. Seq2Seq

강의 주제

1차시: RNN & LSTM: 순환신경망.pdf

- Feed Forward Neural Network
- Time Series Data
- RNN
 - 전통적인 MLP나 CNN은 가변 길이의 입력을 받기 매우 힘들다 (ResNet까지 가야 가변으로 받을 수 있음)
 - RNN은 가변 길이의 입력을 받기 매우 쉬운 구조로 설계됨
 - One to many, Many to one, Many to many
 - Cell
 - Hidden State
 - tanh
 - RNN의 hidden state는 계속 재사용 → state 값이 안정적으로 머물러야 됨
 - tanh는 출력이 $-1 \rightarrow +1$ 이라 그나마 가장 안정적임
 - 그러나 gradient가 0~1 범위라 역시 Vanishing 문제를 일으킬 수 있음 → 밑의 장기 의존성 문제
 - RNN의 Forward Propagation
 - RNN의 Back Propagation
 - RNN의 한계
 - 장기 의존성 문제 (렐루 사용 시 성능 하락 문제 → Gradient Exploding, tanh 대비 렐루가 gradient 컨트롤 하기 더 어려움)

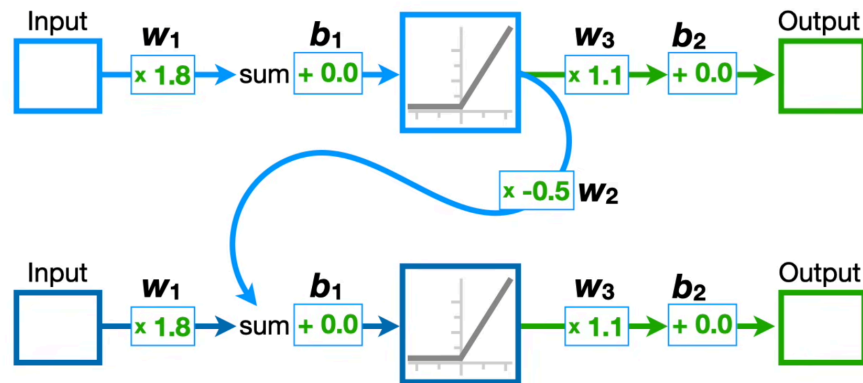
- Gradient Vanishing
- 병렬 처리 어려움
- LSTM
 - Forget Gate
 - Input Gate
 - Hadamard Product
 - Cell State
 - Output Gate
 - Bi-LSTM
- GRU (참고만)

여기까지 RNN 보고서 보면서 복습하세요

2차시: RNN & LSTM: Recurrent Neural Network.pdf (중요하니까 한번 더 볼거임)

1. 7P. → RNN의 기본 개념과 형태

RNN의 Unfold된 형태 (같은 모델이 계속 반복되는 형태)



데이터가 2개일때의 RNN Unfold 형태

2. 19P. ~ 26P. → RNN의 구조

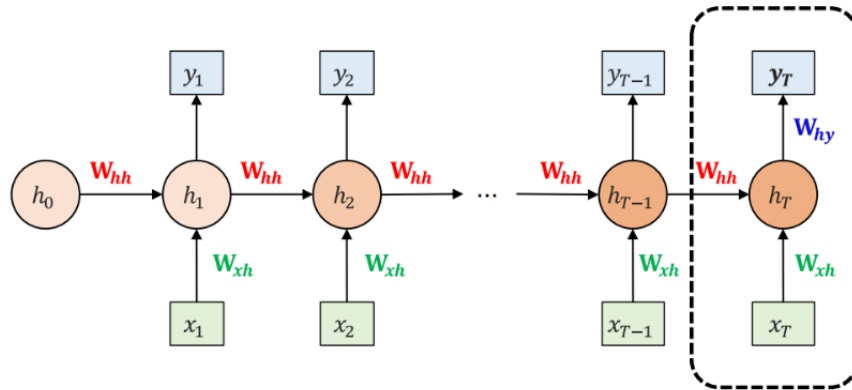
Many to one 형태는 최종 h_t 의 Output만 살림.

그래서 개념적으로는 최종 h_t 일때만 Output 레이어를 붙이는건 맞는데, 실제 코드상에서는 편의상의 이유로 모든 h_t 에 대해 Output을 계산하게 됨

반면 One to many 형태는 최초 h_t 에서만 인풋을 넣음

매번 동일한 인풋을 넣는 형태의 설계도 가능하긴 하지만, 대부분은 최초 1회만 인풋을 넣음

3. 27P. ~ 51P. → RNN의 학습

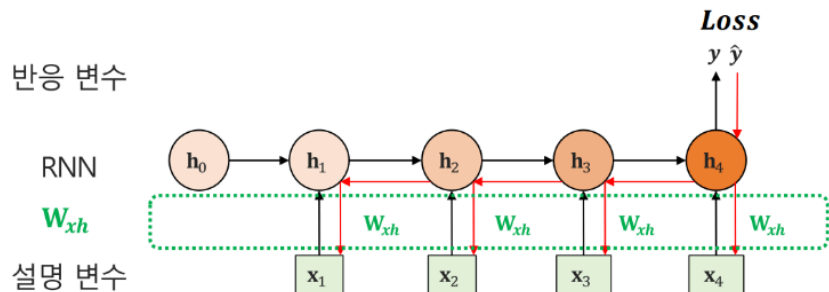


학습의 대상(parameters): (W_{xh} , W_{hh} , W_{hy})

RNN이라는 신경망 구조는 1게임! 그거 가지고 계속 돌리면서 학습하는거임
 $W_{\{hh\}}$ 는 다 똑같은 값임 (다 같은 신경망이니까)

Recurrent 시 weight, 입력에 곱해지는 weight, 출력이 나올 때 곱해지는 weight 이 세개가 학습의 대상이 될 거임

- Loss 계산 == Forward Propagation → 32P. ~ 34P.
 만약 회귀 문제 풀고 싶으면 MSE 사용
 분류라면 CEE 사용
 순전파이므로 $h_1 \rightarrow h_t$ 까지 모든 step을 계산한다
- Gradient 계산 == Back Propagation → 35P. ~ 40P.
 Backward Propagation Through Time (BPTT)를 계산해야 함



$$\frac{\partial Loss}{\partial W_{xh}} = \frac{\partial Loss}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_4} \frac{\partial h_4}{\partial W_{xh}} + \frac{\partial Loss}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial W_{xh}} + \frac{\partial Loss}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{xh}} + \frac{\partial Loss}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{xh}}$$

위와 같은 과정을 h_1 까지 반복, 그 다음 미분값들을 모두 더함
 그 값을 BPTT라고 부름

- Parameter (W_{xh} , W_{hh} , W_{hy}) 학습 → 41P. ~ 42P.
 이전 단계에서 구한 각 파라미터별 BPTT에 Learning Rate를 곱해서 새로운 weight로 업데이트함

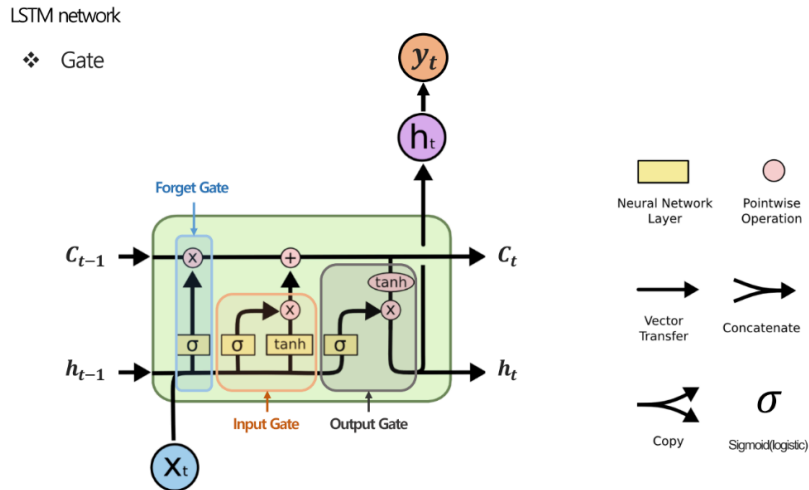
4. 52P. ~ 57P. → RNN의 한계

- RNN은 긴 시퀀스 처리가 어려움 ⇒ LSTM으로 해결
 Activation Function으로 주로 tanh를 쓰는데, tanh의 미분값은 최대가 1이므로 입력되는 시퀀스의 길이가 길수록 (순환이 많을수록) Gradient가 0에 수렴하게 됨
 이를 해결하기 위해 ReLU를 쓰자니 애는 gradient가 양수일 경우 gradient가 기하급수적으로 커지는 gradient exploding 문제가 있음

- RNN은 병렬 처리가 어려움 $\Rightarrow$ LSTM으로 해결 못함, Transformer로 해결
RNN은 결국 하나의 모델을 반복해서 돌리는 구조임
근본적인 구조의 한계 때문에 h_{t+1} 시점의 순전파는 무조건 h_t 시점 이후에 시행되어야 함.
이는 병렬적인 처리를 불가능하게 만들어 학습 시간이 길어지게 함

5. 58P. ~ 77P. $\rightarrow$ LSTM

- Cell State & Gates $\rightarrow$ 59P.
State가 오래 흘러도 정보를 까먹지 않도록 컨베이어 벨트처럼 박아놓음
Cell State에서 잊을건 잊고, 추가할건 추가하자 $\Rightarrow$ Forget / Input Gate
Cell State에서 Hidden State로 얼마나 가져갈 지 정해보자 $\Rightarrow$ Output Gate



- 각 게이트를 목적에 맞게 설계한 것이 아님.
이런 구조를 만들었더니 각 게이트가 이렇게 동작하더라 $\rightarrow$ 그럼 Forget Gate라고 이름을 짓자

3차시: Seq2Seq: Seq2Seq.pdf

- Sequence to Sequence 모델은 시퀀셜 데이터를 받아서 시퀀셜 데이터를 출력하는 구조임
시퀀셜(Sequential) 데이터: 데이터의 순서가 의미를 가지는 데이터 (시계열, 텍스트 등)
우리는 이 중 텍스트 데이터를 주로 다룰거임 (자연어처리로 연결됨)
- 목표: 시퀀셜 $\rightarrow$ 시퀀셜

(영어 $\leftrightarrow$ 독일어 번역기를 만든다고 해봅시다)

문제: Input과 Output 간의 길이가 다르면? & Input과 Output 이 일대일로 매칭이 안되면?

🇬🇧 I don't know. $\Rightarrow$ 🇩🇪 Ich weiß es nicht.

$\rightarrow$ 길이 다른데? (3단어 vs 4단어)

🇬🇧 I was going to call you, but it completely slipped my mind

(직역: 전화하려고 했는데, 그것이 완전히 마음에서 미끄러졌어)

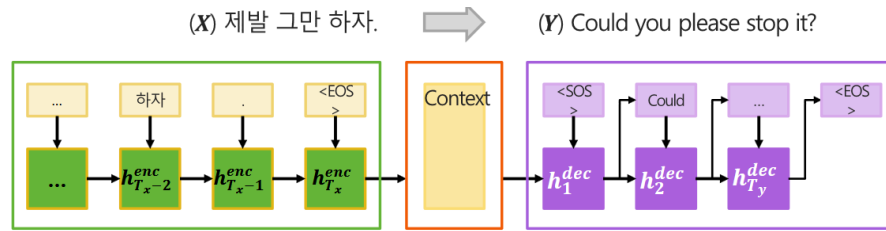
$\Rightarrow$

🇩🇪 Ich wollte dich anrufen, aber ich habe es völlig vergessen.

(직역: 전화하려고 했는데, 나는 그것을 완전히 잊었어)

$\rightarrow$ 뜻이 일대일로 매칭이 안되는데?

- 6P. ~ 10P. $\rightarrow$ Seq2Seq의 구조



한국어를 영어로 번역하는 모델임

입력된 한국어를 요약해서 문맥만 남기는 부분 (초록색)은 **Encoder** 라고 부름

그리고 그 문맥을 풀어서 영어로 재구성하는 부분(보라색)은 **Decoder** 라고 부름

밑에 진한 초록/보라색은 LSTM 또는 GRU 셀을 사용함.

둘다 RNN 계열이라 hidden state가 존재하는데, 문장의 맨 마지막 셀의 hidden state에는 문장 속 모든 토큰들의 정보가 들어있을거임
그걸 Context Vector라고 부름

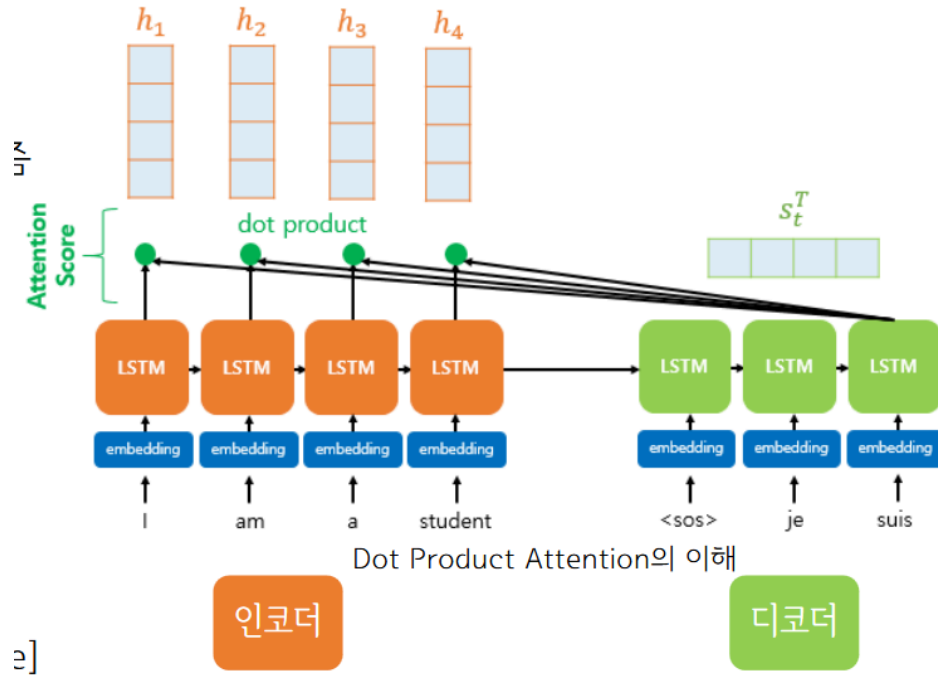
- 11P. ~ 14P. → Context Vector의 문제점
그런데 Context Vector는 고정된 크기 아님?
거기에 문장의 모든 단어를 우겨넣으니 정보가 좀 손실될듯 ⇒ 이걸 bottleneck problem이라 함
또한 그냥 RNN 썼으면 Gradient Vanishing 문제도 있을 수 있을거같음
긴 글을 인코더에 넣으면 RNN처럼 장기 의존성 문제가 발생할 수 있을 듯 하다

4차시: Attention: GAN + Transformer 보고서.pdf 108P. ~ 114P.

- 해결 아이디어: 맨 마지막 hidden state만 주지 말고 그냥 전체 hidden state를 디코더에게 던져주자
문제: 그럼 매 순간 어떤 토큰에 집중할지 어떻게 아냐?
해결책: Attention Mechanism
 - 108P. ~ 109P. → Attention이 뭐임?
말 그대로 중요한 토큰에 집중하게 하는 메커니즘임
Attention은 Query, Key, Value로 구성됨
 - Query: 검색어
모델이 현재 알고자 하는 정보를 나타냄 (검색할때 검색어라고 이해하면 됨)
이는 디코더의 t 시점의 Hidden State와 같음.
(디코더가 t 시점에서 출력할 때 가장 알고자 하는 정보는 t 시점의 Hidden State에 있겠지?)
 - Key: 책 제목
Key에서 Query가 검색함.
이는 모든 시점의 인코더의 Hidden State와 같음.
(출력할 때 정보를 검색하려면 당연히 인코더의 셀들을 뒤져야겠지?)
 - Value: 책 내용
Key랑 정확히 같음.
- 이 Attention에서는 Key와 Value가 같은데, 항상 그런건 아님. (어텐션 종류마다 조금씩 다름)
어텐션은 쿼리와 각 키들 간의 유사도를 계산해서 Attention Score 라는 걸 산출함
1. Query와 각 Key 간의 유사도를 계산함시다
⇒ 이게 바로 Attention Score 임
 2. 이 Attention Score에 Softmax를 취해서 다 더하면 1이 되게 함시다
⇒ 즉, 확률과 동일하게 만든단 소리임. 어텐션 스코어가 여러개니까 각 키가 선택될 확률에 대한 분포 그래프가 그려지겠지? → 이걸 Attention Distribution이라 함
 3. Attention Distribution과 각 Value를 곱한 다음 더함시다
⇒ 이게 바로 Attention Value 임

정리: Attention Score → Attention Distrubution → Attention Value

- 110P. ~ 114P. → Dot-Product Attention (Luong Attention 이라고도 부름)



천천히 쿼리 키 밸류부터 살펴봅시다

- Query
디코더의 hidden state = s_t
- Key
모든 시점의 인코더 셀의 hidden state = $[h_1, h_2, h_3, \dots]$
- Value
모든 시점의 인코더 셀의 hidden state = $[h_1, h_2, h_3, \dots]$ (Key와 동일)

목표는?

⇒ t 시점에서 단어 출력하기 ⇒ t 시점에서 가장 확률이 높은 출력값 고르기 ⇒ **t 시점에서 가장 높은 Attention Value** 구하기

1. 111P. ⇒ Attention Score 구하기
쿼리와 키의 유사도를 계산해야 한다.
행렬 간의 유사도는 서로 곱하면 됨
쿼리 (s_t)와 키 $[h_1, h_2, h_3, \dots]$ 를 곱해줍니다
결과: $s_t^T h_1 \ s_t^T h_2 \dots s_t^T h_i \Rightarrow e^t$ 라 합니다.
(T는 Transpose로, 계산을 위해 모양 맞추는 과정이라 생각하면 됨)
2. 112P. ⇒ Attention Distribution 구하기
이 e^t 라는 벡터를 확률분포로 나타내야 함.
Softmax를 적용해봅시다.
모든 값을 합하면 1이 되는 분포를 얻어낼 수 있음
3. 113P. ⇒ Attention Value 구하기
분포와 밸류를 곱한 다음 더해야 함
각 인코더의 hidden state와 Attention Weight (Attention Distribution 그래프의 개별적인 값)를 각각 곱한 다음 모두 더합니다
4. 114P. ⇒ 디코더가 다음 단어를 생성할 때 보라고 던져줍니다
그런데 Attention Value만 보내면 안됨
디코더가 지금까지 생성하면서 가지고 있던 정보도 같이 던져주는게 맞겠지? ← RNN이니까

정확히는, RNN/LSTM이 순환하면서 이전 hidden state를 입력으로 받을건데 그 입력에다가 우리가 구한 Attention Value를 곱해서 끼울거임

따라서 Attention Value와 디코더의 Hidden State를 Concatenate(이어붙이기) 합시다
그리고 그걸 디코더에게 던져넣으면 끝!

- 이럼 뭐가 좋나요?
 - Attention: 집중 가능
디코더가 입력 전체가 아니라 필요한 부분에만 집중할 수 있게 됨
 - Bottleneck Problem 해결
디코더가 인코더의 모든 Hidden State에 접근할 수 있음
 - Gradient Vanishing / 장기 의존성 문제 완화 가능
Attention은 멀리 떨어진 단어도 의무적으로 모두 계산함

5차시: Transformer: 트랜스포머.pdf

- Self Attention
이전의 Luong Attention은 인코더 ↔ 디코더 간을 이어주는 어텐션이었음.
즉, 입력문장 ↔ 출력문장을 이어주는 놈임
만약 하나의 문장 안에서 어텐션을 쓴다면? ⇒ "Self" Attention

Luong은 출력 문장을 만들기 위해 입력 문장의 어디를 봐야 하는가? 에 대한 답이었음
근데 어텐션은 결국 많은 정보 중 나에게 중요한 정보만 집중하는거임

The animal didn't cross the street because **it** was tired.
위 문장에서 it이 의미하는 단어는? ⇒ animal에 집중해야됨

⇒ 이걸로 트랜스포머를 공부해 봅시다

- 10P. 오버뷰
입력이 들어오면 아래와 같은 과정을 거쳐 예측된 단어를 뱉어냄
 1. 입력 전처리: Tokenization & Embedding
 2. 입력 전처리: Positional Encoding
 3. Multi-Head Attention → MHA
 4. Encoder
 5. Masked MHA
 6. Decoder
 7. 단어 예측 (출력)이걸 하나하나 살펴봅시다
- 11P. ~ 12P. → Tokenization & Embedding
 - Tokenization
→ 텍스트 잘게 쪼개기 (설명 끝)
 - Embedding
→ 잘게 자른 토큰들 (텍스트)을 숫자로 바꾸기 (설명 끝)
- 13P. ~ 14P. → Positional Encoding
트랜스포머는 문장을 쪼개 토큰을 만든 후 그걸 한꺼번에 병렬로 처리함
그래서 위치 정보가 소실되는 문제가 있음

A가 B를 좋아한다.

B가 A를 좋아한다.

→ 순서만 바뀌었는데 전혀 다른 뜻이 됨

이러면 안되므로 각 토큰마다 이거 문장의 몇 번째 토큰이다 라고 위치정보를 더해줌

(토큰 리스트에서 인덱스를 매긴거랑 유사한 느낌임)

실제 트랜스포머는 이를 $\sin$, $\cos$ 로 굳이굳이 복잡하게 처리함

사인과 코사인을 복잡하게 이리저리 섞으면 각 위치마다 고유한 값이 만들어진다

게다가 위치 정보 뿐만 아니라 두 토큰들 사이 상대적인 위치까지도 알 수 있는 방법이라 이렇게 한다 함

- 15P. ~ 22P. → MHA

참고: Transformer 보고서 131P.

그냥 하나의 Self Attention보다 같은 입력을 여러 어텐션에 동시에 쏘서넣고 결과를 합치는게 더 성능이 좋았다고 함

입력을 똑같이 넣고, 각 Self Attention들이 알아서 자기만의 Q, K, V 를 만들. (이 때 쓰이는 Self Attention들은 Head라고 부름)

그런 다음 출력값들을 Concatenate 후 Linear 함수를 거쳐서 사용함 (당연히 병렬로 Head들을 한번에 계산함)

헤드가 멀티로 있으므로 Multi-Head Attention이라 부름

비유하자면 같은 문장을 입력으로 줬을 때 1번 Head는 단어의 문맥적 관계를 보고, 2번 Head는 단어의 시제를 보고, 3번 Head는 문장 속 명사들을 보는 느낌임

(몇번 Head가 어떤 것에 Attention 했는지는 모름)

- 23P. ~ 24P. → Encoder

ResNet의 Residual Block처럼 입력값을 그대로 출력으로 싸줌시다

이렇게 하면 ResNet의 장점처럼 학습이 조금 더 안정적으로 이루어짐

또한 Layer Normalization이라는 걸 함

학습을 하면서 hidden layer 의 weight들이 커졌다 작아졌다 할텐데 이럼 학습이 좀 불안정해짐

그래서 한번 정렬하고 가는거임

$[0, 1, 4, 2, 6, 10] \rightarrow [0, 0.1, 0.4, 0.2, 0.6, 1]$ 이런 느낌으로 변환시키는게 정규화(Normalization) 임

- 25P. ~ 26P. → Decoder

참고: Transformer 보고서 133P.

인코더와 동일하게 Self Attention을 이용해서 Attention Value를 계산해줍니다

그런데 이 때는 미래의 정보를 보면 안됨 (출력이니까 당연함)

이렇게 해야 단어를 출력할 때 이전에 생성된 단어들만을 기반으로 다음 단어를 예측하는 특성을 유지할 수 있음

이를 위해 여기서는 그냥 MHA를 쓰지 않고, 미래 단어를 제거한 Masked MHA를 사용함

디코더는 아래 Masked MHA, 위쪽 그냥 MHA로 나뉜다

- 아래쪽 부분

여기는 인풋으로 정답을 받는다.

기본적으로 이 모델 아키텍처는 트랜스포머 학습 할 때 그림임

당연히 실제로 쓸 때나 테스트 때는 정답이 아니라 다른걸 넣음 (보통 <sos> 가 들어감)

정답으로 들어온 단어를 앞에서부터 봐가면서 문장의 문맥을 파악해야 하는데, 문장을 한 번에 다 보면 컨닝임 → 미래 단어 못보게 마스크 → Masked MHA

- 위쪽 부분

여기는 인풋으로 인코더와 Masked MHA의 출력값을 받는다

여기 쓰이는 어텐션은 Cross Attention이라 부름 (Encoder 와 Decoder를 이어주는 역할이므로)

Luong Attention과 매커니즘은 거의 비슷함

내일 오후 라이브 추가내용 (참고하세요)

- MobileNet

핵심 차이: 여러 채널에 대해 한번에 컨볼루션 연산하는거까진 똑같은
근데 각 채널별로 나온 컨볼루션 값을 합하지 않고 그대로 사용함

일반 conv layer: 3채널 * 커널 컨볼루션 연산 → 1채널로 압축

모바일넷의 Depthwise 방식: 3채널 * 커널 컨볼루션 연산 → 3채널 그대로 출력됨

- Vision Transformer

- 원본 트랜스포머는 텍스트를 받도록 설계됨
- 반면 애는 이미지를 받도록 설계됨
- 원본은 텍스트를 토큰화해서 잘게 쪼개 사용하지만, 이미지는 토큰화하기 좀 그러니 대신 격자로 잘게 자름 → 이걸 token 대신에 patch라 부를거임
- 그 patch들을 원본 트랜스포머와 동일한 과정을 거쳐 어텐션하고 FC 레이어 통과시키고 최종 출력으로 [CLS] 토큰을 출력함
- 원본 논문에서는 [CLS] 토큰은 입력된 이미지가 된 사진인지에 대한 정보를 담고 있음
따라서, 비전 트랜스포머는 이미지가 된 사진인지 분류를 수행할 수 있게 됨
- 반드시 이미지 분류만 수행할 수 있는건 아님.
대표적인 예시가 이미지 분류인거지 객체 탐지, 이미지 생성, 이미지 설명 출력 등 Vision과 관련된 작업을 수행할 수 있음